



中山大學

SUN YAT-SEN UNIVERSITY

软件工程学院

SCHOOL OF SOFTWARE ENGINEERING



SSE206 Computer Networking

Chapter2-Application Layer

Yuhong Nan

nanyh@mail.sysu.edu.cn

2026-03-16

应用层 Application layer: overview



- 应用层协议原理 Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS 域名解析
- P2P applications
- video streaming and content distribution networks 视频流和内容分发网
- socket programming with UDP and TCP 套接字编程



应用层 Application layer: overview



Our goals:

■ 网络应用原理：网络应用架构

- 客户-服务器模式
 - client-server paradigm
- P2P 对等模式
 - peer-to-peer paradigm
- 传输层所提供的服务模型

■ 网络应用的实例：互联网流行的应用层协议

- HTTP
- 电子邮件 SMTP
- DNS
- P2P文件分发
- 视频流应用, CDN 内容分发网

■ 编程：网络应用程序

- socket API

一些网络应用的例子



- 社交网络 social networking
- 直播/短视频 TikTok
- 文本消息 text messaging
- 邮件 e-mail
- 多人网游 multi-user network games
- 流媒体 streaming stored video (YouTube, Hulu, Netflix)
- P2P文件共享 P2P file sharing
- 实时视频会议 real-time video conferencing (e.g., Zoom)
- MQTT (Message Queuing Telemetry Transport)
- ...

创建一个网络应用

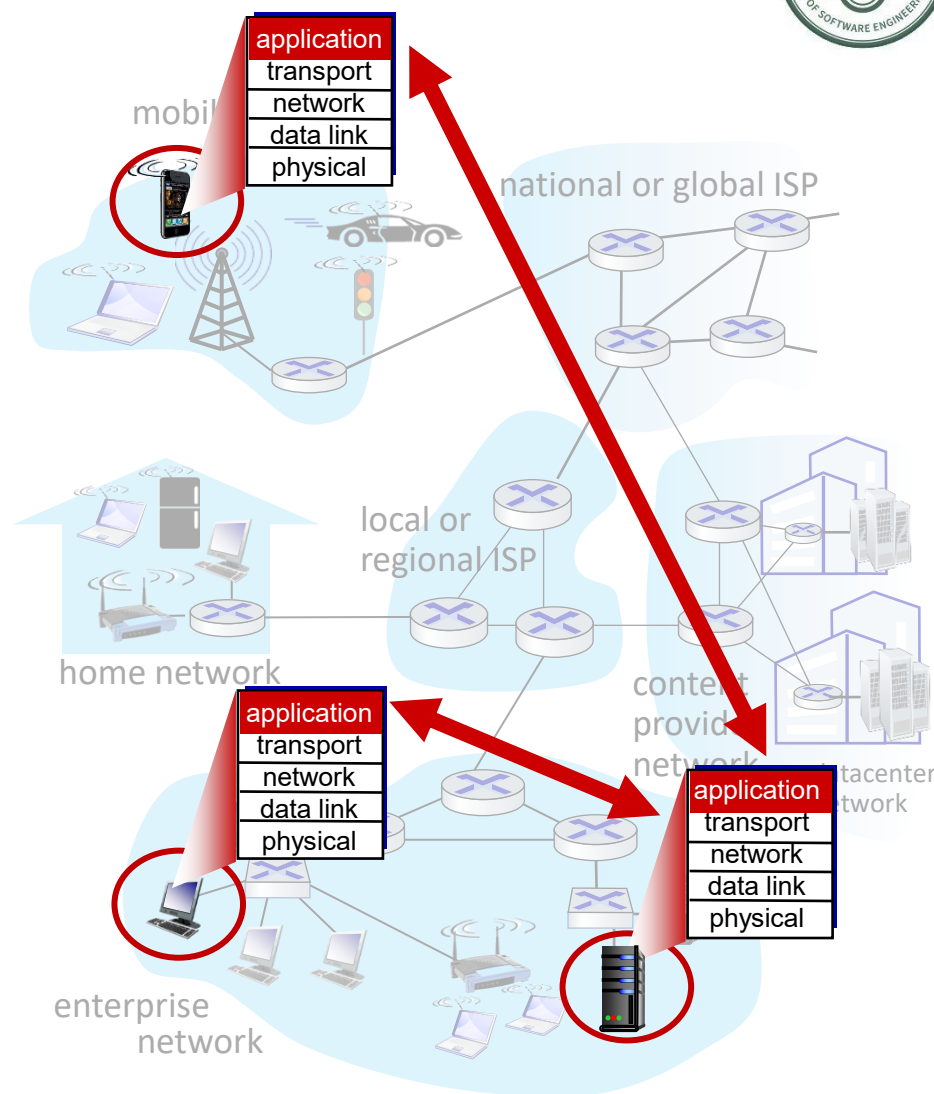


创建这样的程序

- 在不同的端系统上运行
- 通过网络基础设施提供的服务，实现彼此通信
 - 例如，Web 服务器软件与浏览器软件通信

网络核心中没有应用层软件

- 网络核心没有应用层功能
- 网络应用只在端系统上存在，因此可以支持快速开发和部署



客户-服务器 (C/S) 体系结构

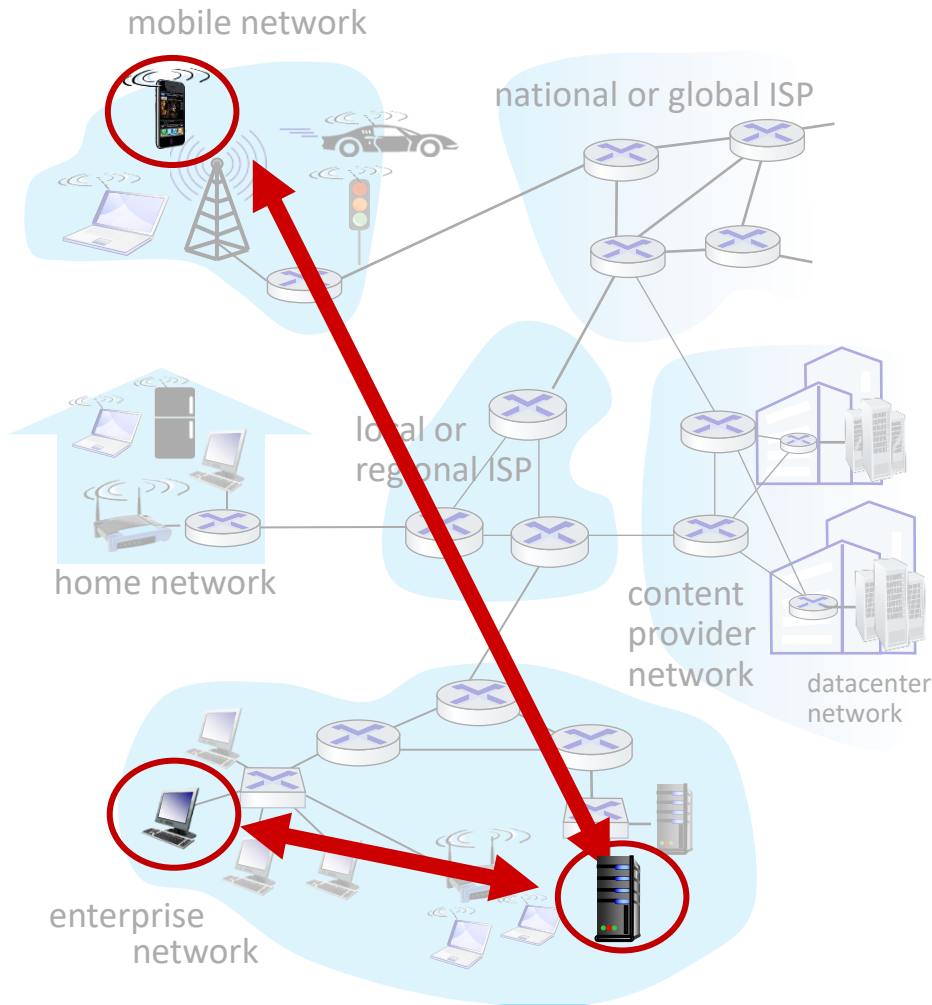


服务器 server:

- 一直运行的主机 always-on host
- 固定的IP地址和端口号 (约定)
- 为了支持大规模、扩展性, 通常位于数据中心

客户端 clients:

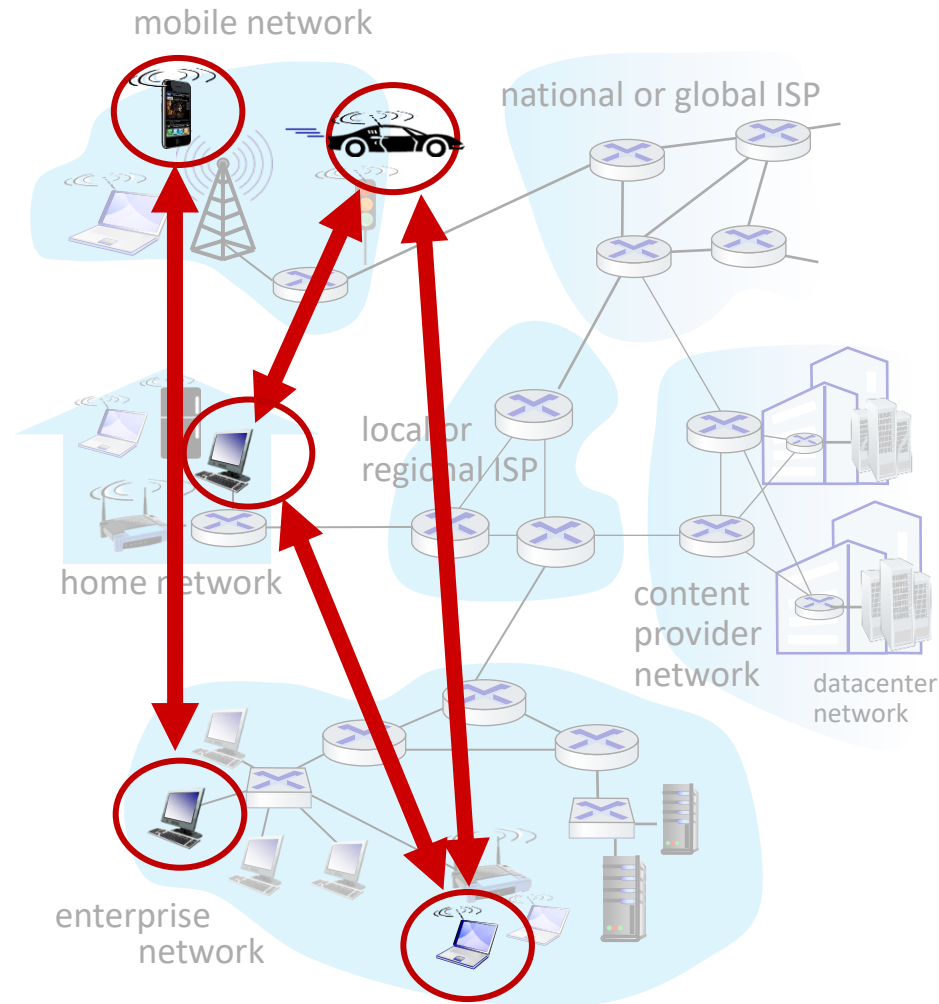
- 主动与服务器通信
- 与互联网有间歇性的连接
- 可能是动态IP地址
- 不直接与其它客户端通信
 - 示例: HTTP, IMAP(Internet Message Access Protocol), FTP(File Transfer Protocol)



P2P 体系结构



- (几乎) 没有一直运行的服务器
- 任意端系统之间可以进行通信
- 每个节点 既是客户端又是服务器
 - 自扩展性: 新节点 (peer) 带来新的服务能力, 当然也带来新的服务请求
- 参与的主机**间歇性连接**且可以改变IP 地址
 - 管理复杂
 - 举例: P2P 文件共享 – Windows Update



C/S和P2P体系结构的混合体



■ Napster

- 文件搜索 – C/S文件搜索：集中式
- 文件共享 – P2P文件传输：P2P式

■ Instant messaging 即时通信

- Online detection 在线检测：集中式
 - 当用户上线时，向中心服务器注册其IP地址
 - 用户与中心服务器联系，以找到其在线好友的位置
- 两个用户之间聊天：P2P（可能的一种实现方式）

Processes communicating 进程通信



进程Process: 在主机上运行的应用程序

- 在同一个主机内, 使用进程间通信机制 IPC 进行通信 (操作系统定义)
- 不同主机, 基于应用协议, 通过交换报文 (message) 来通信

clients, servers

客户端进程client process:
发起通信的进程

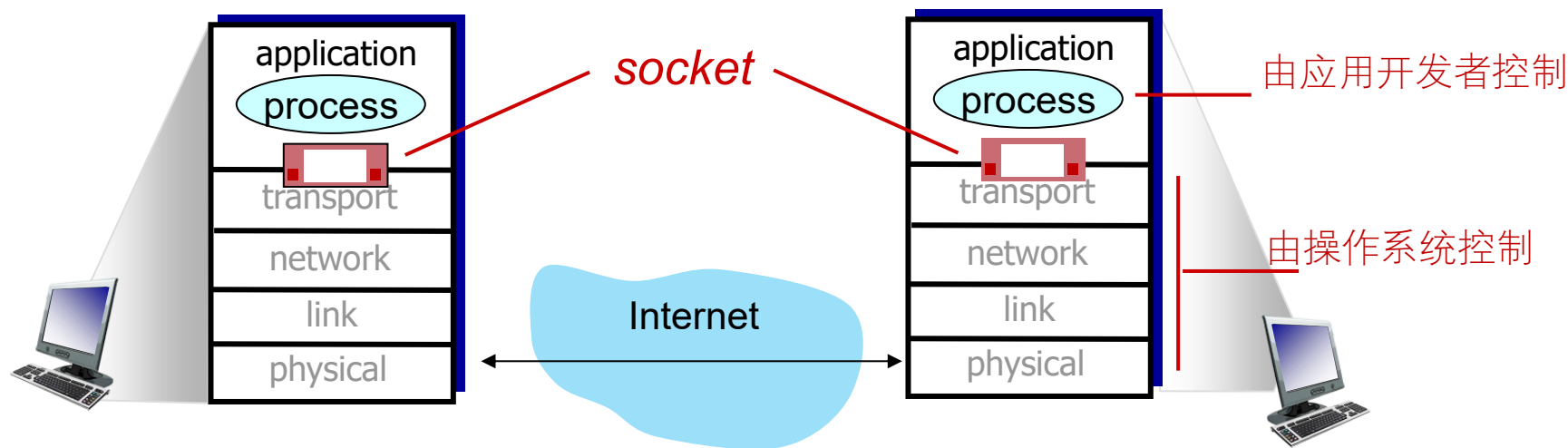
服务器进程server process:
等待连接的进程

- 注意: P2P架构的应用也有客户端进程和服务端进程之分

Socket 套接字



- 进程向套接字(socket)发送报文，或从套接字接收报文
- 类似于传送门
 - 发送进程将报文推出门户
 - 发送进程依赖于传输层设施在另外一侧的门，将报文交付给接收进程
 - 接收进程从另外一端的门户收到报文（同样依赖于传输层设施）



Addressing processes 进程寻址



- 要接收消息，进程必须具有标识符 **identifier**
 - 主机设备有唯一的32位IP地址
- **Q:** 进程所在主机的IP地址是否足以识别该进程?
- **A:** 不，许多进程可以在同一台主机上运行
- 标识符 **identifier** 包括
 - 与主机上进程关联的 **IP地址**
 - **端口号 port number**。
- 端口号示例:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - **IP address:** 128.119.245.12
 - **port number:** 80
- 端口号: 16 bit 比特 (0-65535)

应用层协议



■ 交换的信息类型

- e.g., 请求request, 响应response

■ 消息的语法 syntax :

- 消息中的哪些字段&如何描述字段

■ 消息语义 semantics:

- 字段中信息的含义

■ 进程何时以及如何发送和响应消息的规则

■ 公开协议 open protocols:

- 由RFC文档定义
- 允许共享、互用
- e.g., HTTP, SMTP

■ 专用（私有）协议 proprietary protocols:

- e.g., Wechat, Whatsapp, Zoom

应用需要传输层提供什么样的服务



可靠性 reliability

- 有些应用则要求100%的可靠数据传输（如文件）
- 有些应用（如音频）能容忍一定比例以下的数据丢失

定时保证 timing

- 一些应用出于时效性考虑，对数据传输有严格的时间限制

吞吐量 throughput

- 一些应用（如多媒体）必须需要最小限度的吞吐量，从而使得应用能够有效运转
- 一些应用能充分利用可供使用的吞吐(弹性应用)

安全性 security

- encryption, data integrity, ...机密性、完整性、可认证性

常见应用对传输服务的要求



应用	数据丢失率	吞吐	时间敏感性
文件传输	不能丢失	弹性	不
e-mail	不能丢失	弹性	不
Web 文档	不能丢失	弹性	不
实时音视频	容忍丢失	音频: 5kbps-1Mbps 视频: 0kbps-5Mbps	是, 100ms
存储音视频	容忍丢失	同上	是, 几秒
交互式游戏	容忍丢失	几kbps ~10kbps	是, 100ms
即时讯息	不能丢失	弹性	是和不是

常见应用对传输服务的要求



当前主流网络应用需求特征（2024–2025）

应用	数据丢失率	典型吞吐需求	时间敏感性
短视频流媒体 TikTok / YouTube Shorts	容忍丢失	2–25 Mbps（1080p/4K） 自适应码率 ABR	缓冲约 2–5 s
实时视频会议 Zoom / Teams / Meet	容忍丢失	1–4 Mbps/人（720p） 音频 40–100 kbps	≤ 150 ms RTT
云游戏 Xbox Cloud / GeForce NOW	容忍少量	10–35 Mbps（1080p） 4K 可达 100 Mbps	≤ 40 ms RTT
在线 AI 推理 / LLM ChatGPT / Copilot / API	不能丢失	弹性（文本 kbps 级） 流式输出 1–30 tokens/s	TTFT ≤ 1–2 s
短消息 / 即时通讯 微信 / WhatsApp / Slack	不能丢失	弹性（极低带宽） 文字 < 1 kbps；图片 MB 级	是，感知 < 1 s
大文件传输 / 云备份 iCloud / OneDrive / S3	不能丢失	弹性，尽量占满带宽 TB 级任务可跑数天	不敏感

*Prompt: 参考原图格式，对当前主流、实际的集中不同应用的数据丢失率、吞吐、时间敏感性进行说明。

Internet 传输层提供的服务



TCP service:

- 可靠的传输服务 reliable transport
- 流量控制 flow control: 发送方不会淹没接受方
- 拥塞控制 congestion control: 当网络出现拥塞时，能抑制发送方
- 不能提供的服务: 时间保证、最小吞吐保证和安全
- 面向连接 connection-oriented: 要求在客户端进程和服务端进程之间建立连接

Internet 传输层提供的服务



TCP service:

- 可靠的传输服务reliable transport
- 流量控制 flow control: 发送方不会淹没接受方
- 拥塞控制 congestion control: 当网络出现拥塞时, 能抑制发送方
- 不能提供的服务: 时间保证、最小吞吐保证和安全
- 面向连接connection-oriented: 要求在客户端进程和服务端进程之间建立连接

UDP service:

- 不可靠数据传输unreliable data transfer
- 不提供的服务: 可靠, 流量控制、拥塞控制、时间、带宽保证、建立连接

Q: 为什么要有 UDP?

UDP存在的必要性



- **能够区分不同的进程，而IP服务不能**
 - 在IP提供的**主机到主机**功能的基础上，区分了主机的应用进程
- **无需建立连接，适合事务型的应用**
- **不做可靠性的工作，例如检错重发，适合对实时性要求较高而对正确性要求不高的应用**
 - 为了实现可靠性（准确性、保序等），必须付出时间代价（检错重发）
- **没有拥塞控制和流量控制，应用能够按照设定的速度发送数据**
 - 而在TCP上面的应用，应用发送数据的速度和主机向网络发送的实际速度是不一致的

Internet应用及其应用层协议和传输协议



应用	应用层协议	下层传输协议
电子邮件	SMTP [RFC 5321] IMAP [RFC 9051] 原 RFC 2821 已由 5321 取代	TCP 端口 25 / 587 / 993
远程访问	SSH [RFC 4251] (已全面取代 Telnet) Telnet 明文传输, 现已弃用	TCP 端口 22
Web 浏览	HTTP/2 [RFC 7540] HTTP/3 [RFC 9114] 原 HTTP/1.1 RFC 2616 已废止	TCP (HTTP/2) QUIC / UDP (HTTP/3) 端口 443 (TLS)
文件传输	SFTP / SCP (基于 SSH) FTPS [RFC 4217] 裸 FTP 已基本淘汰	TCP 端口 22 / 990
流媒体 点播 / 直播	HLS [RFC 8216] DASH (ISO 标准) RTMP (直播推流) RealNetworks 专有协议已退出市场	TCP / QUIC 基于 HTTP(S) 分段下载
VoIP / 网络电话	SIP [RFC 3261] (信令) RTP / SRTP [RFC 3550] (媒体) Net2Phone 专有协议已淘汰	UDP (媒体流) TCP / UDP (SIP)

Securing TCP 安全TCP



TCP & UDP sockets:

- no encryption 都没有加密
- 明文传输 (包括账号、密码)

Transport Layer Security (TLS)

- 在TCP上面实现, 提供加密的TCP连接
- 数据完整性 data integrity
- 端到端的认证 end-point authentication

在应用层实现

- 应用程序使用SSL/TLS库, 这些库又使用TCP
- 发送到“套接字”的明文经过加密后在互联网传输
- more: Chapter 8



Application layer: overview



- Principles of network applications
- **Web and HTTP**
- E-mail, SMTP, IMAP
- The Domain Name System DNS

- P2P applications
- video streaming and content distribution networks
- socket programming with UDP and TCP



Web and HTTP



First, a quick review...

- Web 网页：由一些对象组成
- 对象可以是HTML文件、JPEG图像、Java小程序、声音剪辑文件等...
- Web 网页含有一个**基本的HTML文件**，该基本HTML文件又包含若干对象的引用（链接）
- 通过URL对每个对象进行引用：访问协议、（用户名，密码）域名、（端口）、路径、文件名等。

`http://www.someschool.edu:80/someDept/pic.gif`

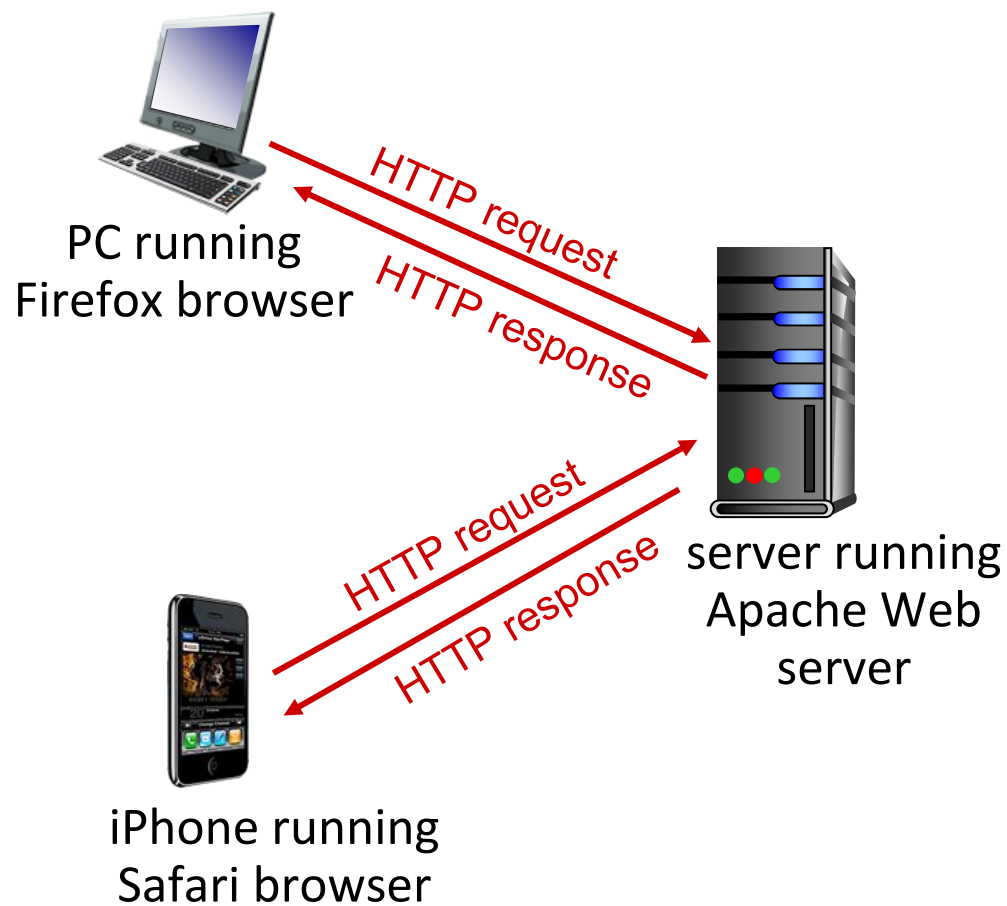
host name

path name

HTTP 概况



- HTTP: hypertext transfer protocol
- 超文本传输协议
- Web的应用层协议
- 客户/服务器模式client/server model:
 - 客户端 client: 请求、接收和显示 Web对象的浏览器
 - 服务器 server:对请求进行响应, 发送对象的Web服务器
- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



HTTP 概况



使用TCP:

- 客户端发起一个与服务器的TCP连接 (建立套接字), 服务器端口号为 80
 - 服务器接受客户端的TCP连接请求
 - 浏览器(HTTP客户端) 与 Web服务器(HTTP服务器)交换HTTP 报文 (应用层协议报文)
 - TCP连接关闭
- HTTP 是无状态的 stateless
 - 服务器并不维护关于客户端的任何信息

aside 维护状态的协议很复杂!

- 必须维护历史信息 (状态)
- 如果服务器/客户端死机, 它们的状态信息可能不一致 (但二者的信息需要保持一致)
- 无状态的服务器能够支持更多的客户端

两类HTTP连接: 非持久/持久



非持久HTTP

- 最多只有一个对象在 TCP连接上发送
- 下载多个对象需要多个TCP连接
- HTTP/1.0使用非持久连接

持久HTTP

- 多个对象可以在一个（客户端和服务端之间的）TCP连接上传输
- HTTP/1.1 默认使用持久连接

非持久HTTP连接: 例子



假设用户输入URL: `www.someSchool.edu/someDepartment/home.index`
(包含文本和10个 jpeg 图像的引用)



1a. HTTP客户端在端口号80发起一个到服务器 `www.someSchool.edu` 的连接



1b. 位于主机 `www.someSchool.edu` 的HTTP服务器在80号端口等待连接, 接受连接并通知客户端

2. HTTP客户端向TCP连接的套接字发送HTTP请求报文, 报文表示客户端需要对象 `someDepartment/home.index`

3. HTTP服务器接收到请求报文, 检索出被请求的对象, 将对象封装在一个响应报文, 并通过其套接字向客户端发送

time
↓

非持久HTTP连接: 例子



假设用户输入URL: `www.someSchool.edu/someDepartment/home.index`
(包含文本和10个jpeg图像的引用)



4. HTTP关闭TCP连接。

5. HTTP客户端收到包含html文件的响应报文，并显示html。然后对html文件进行检查，找到10个jpeg引用对象

6. 对10个jpeg对象，重复1-5步

time



非持久HTTP连接:响应时间模型



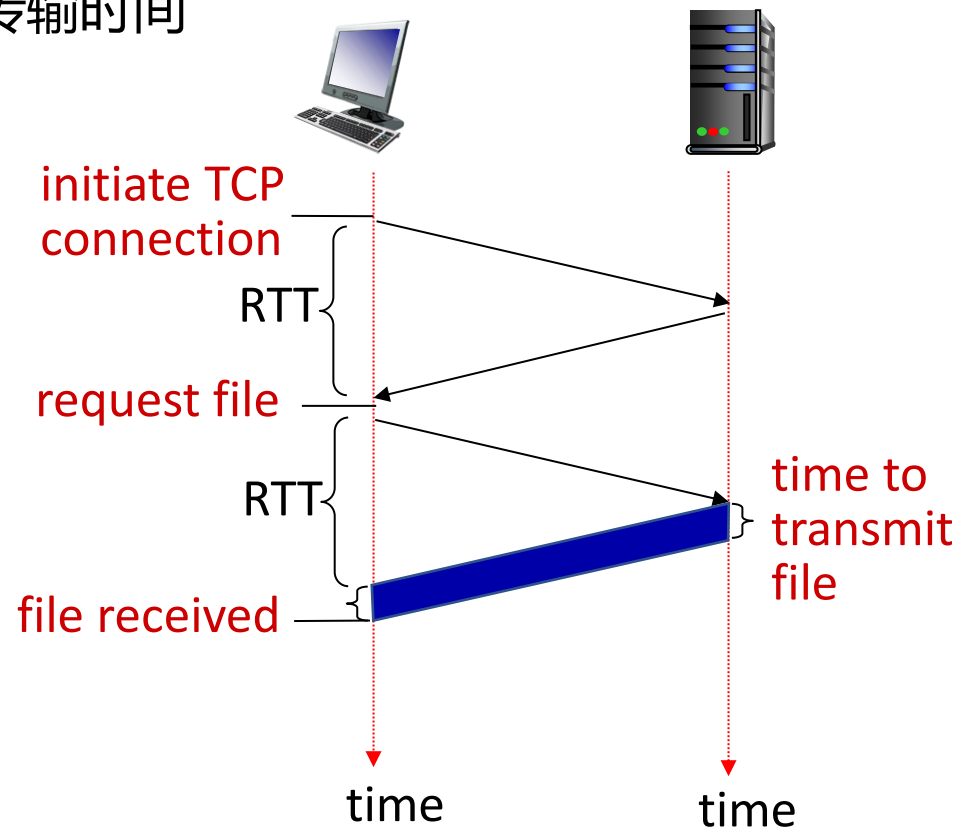
往返时间RTT (round-trip time) :

- 一个小的分组从客户端到服务器，再回到客户端的时间（传输时间忽略）

HTTP响应时间(per object):

- 一个RTT用来发起TCP连接
- 一个RTT用来发起HTTP请求，并等待HTTP响应
- 文件传输时间

非持久HTTP连接响应时间 = $2RTT +$ 文件传输时间



Persistent 持久 HTTP (HTTP 1.1)



非持久HTTP的缺点:

- 每个对象要2个RTT
- 操作系统必须为每个TCP连接分配资源
- 但浏览器通常打开并行TCP连接，以获取引用对象

持久HTTP(HTTP1.1):

- 服务器在发送响应后，仍保持TCP连接
 - Connection: keep-alive
- 相同客户端和服务端之间的后续请求和响应报文，通过同一个连接进行传送
- 客户端在遇到一个引用对象的时候，可以尽快发送该对象的请求

Persistent 持久 HTTP (HTTP 1.1)



非流水方式的持久HTTP:

- 客户端只能在收到前一个响应后才能发出新的请求
- 每个引用对象花费一个RTT

流水方式的持久HTTP:

- HTTP/1.1的默认模式
- 客户端遇到一个引用对象就立即产生一个请求
- 所有引用对象只花费一个RTT是可能的（如很小的文件）

HTTP 请求报文



■ 两种类型的HTTP报文: 请求 Request, 响应 Response

■ HTTP请求报文:

➤ ASCII 格式编码

请求行 (GET, POST, HEAD 命令)

carriage return character
line-feed character

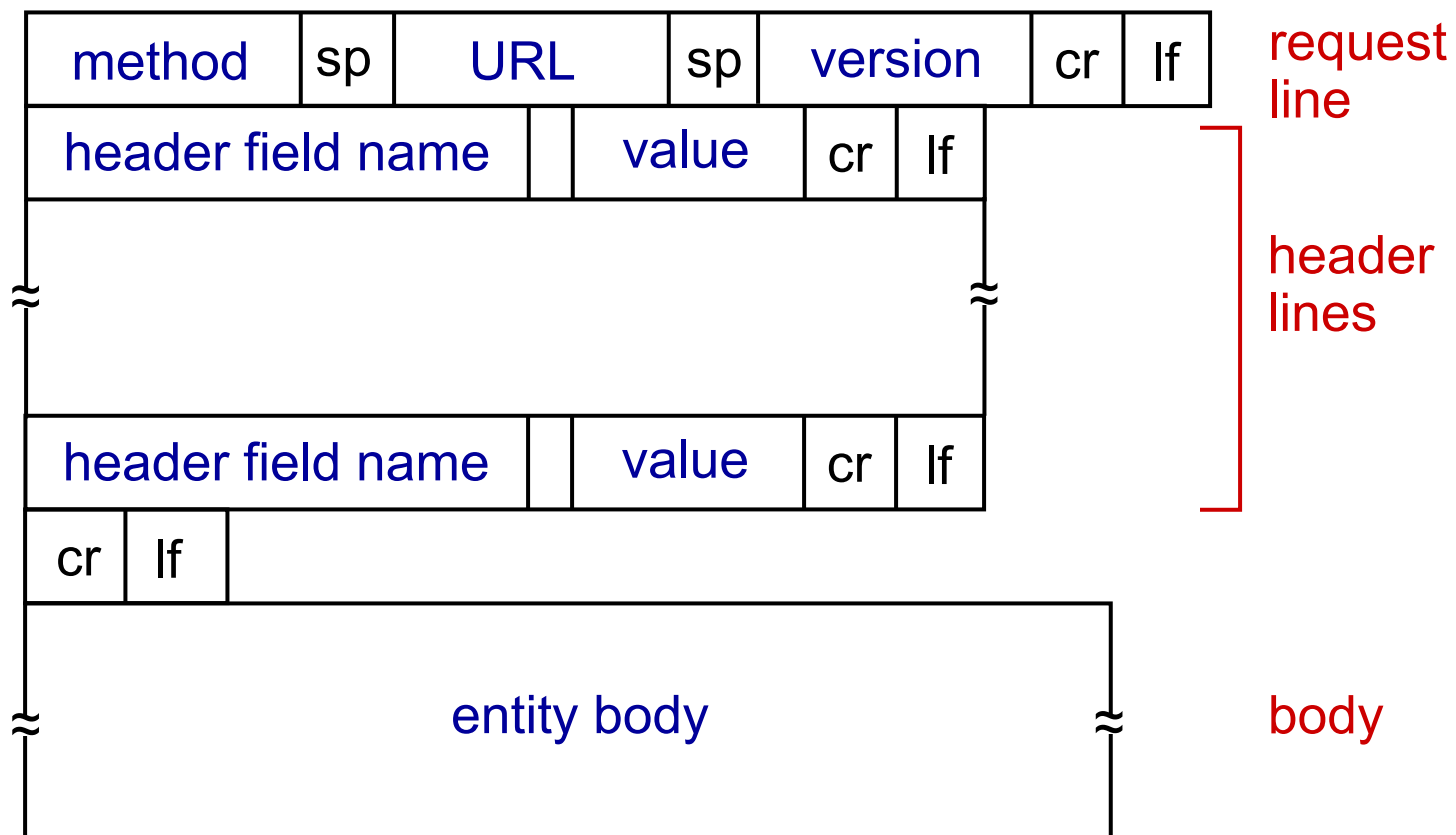
首部行 header lines

换行回车符,
表示报文结束

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0)
          Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

* Check out the online interactive exercises for more examples:
http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP 请求报文：通用格式



提交表单输入



GET 请求 (用于向服务器发送数据):

- 在HTTP GET请求消息的URL字段中包含用户数据(在`?`之后传参)

```
https://s.weibo.com/weibo?q=tictok&xsort=hot&haslink=1&timescope=custom:2022-03-01:2022-03-02&Refer=g
```

热门 ×

含短链 ×

2022-03-01 00时~2022-03-02 23时 ×

POST 请求:

- 网页通常包括表单输入
- 包含在实体主体 (entity body) 中的输入被提交到服务器

HEAD method:

- requests headers (only)

- 不包含请求对象

- 故障跟踪
- 辅助搜索引擎检索

DELETE method:

- 删除URL字段规定的文件.

PUT method:

- 上传新文件(对象)到服务器
- 用POST HTTP请求消息实体中的内容完全替换指定URL上存在的文件

HTTP响应报文



状态行 (协议版本、
和相应状态信息)

状态码

header
lines

HTTP/1.1 200 OK

Date: Tue, 08 Sep 2020 00:53:20 GMT

Server: Apache/2.4.6 (CentOS)

OpenSSL/1.0.2k-fips PHP/7.4.9

mod_perl/2.0.11 Perl/v5.16.3

Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT

ETag: "a5b-52d015789ee9e"

Accept-Ranges: bytes

Content-Length: 2651

Content-Type: text/html; charset=UTF-8

\r\n

data data data data data ...

数据, 如请求的HTML文件

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP响应状态码



- 位于服务器——客户端的响应报文中的首行
- 一些状态码的例子:

200 OK

- 请求成功，请求对象包含在响应报文的后续部分

301 Moved Permanently

- 请求的对象已经被永久转移了；新的URL在响应报文的Location: 首部行中指定
- 客户端软件自动用新的URL去获取对象

304 Not Modified

400 Bad Request

- 一个通用的差错代码，表示该请求不能被服务器解读

404 Not Found

- 请求的文档在该服务上没有找到

用户-服务器状态: cookies



- 回想一下:HTTP GET/response交互是无状态的
- 没有多步HTTP消息交换来完成某项 “事务” 的概念
 - 不需要客户端/服务器跟踪多步交换的 “状态”
 - 所有HTTP请求都是相互独立的
 - 客户端/服务器不需要从部分完成, 但未完全完成的事务中 “恢复”

用户-服务器状态: cookies



- 大多数主要的门户网站使用 cookies

- 四个组成部分:

- 1) 在HTTP**响应报文**中有一个cookie的首部行(set cookie XX)
- 2) 在HTTP**请求报文**含有一个cookie的首部行(cookie = XX)
- 3) 在用户端系统中保留有一个cookie文件, 由用户的浏览器管理
- 4) 在Web站点有一个后端数据库



用户-服务器状态: cookies



Example:

- Susan总是用同一个PC使用IE浏览器上网
- 她第一次访问了一个使用了Cookie的电子商务网站
e.g., Amazon/PDD
 - Unique ID (aka “cookie”)
 - 在后端数据库中输入ID
- 当最初的HTTP请求到达服务器时, 该站点产生一个唯一的ID, 并以此作为索引在它的后端数据库中产生一个项



Cookies: 维护状态



HTTP Cookies: 讨论



Cookies能带来什么:

- 用户验证 Authorization
- 购物车 Shopping carts
- 推荐 Recommendations: Amazon
- 用户状态 User session state (Web e-mail)

Cookies 与用户隐私: aside

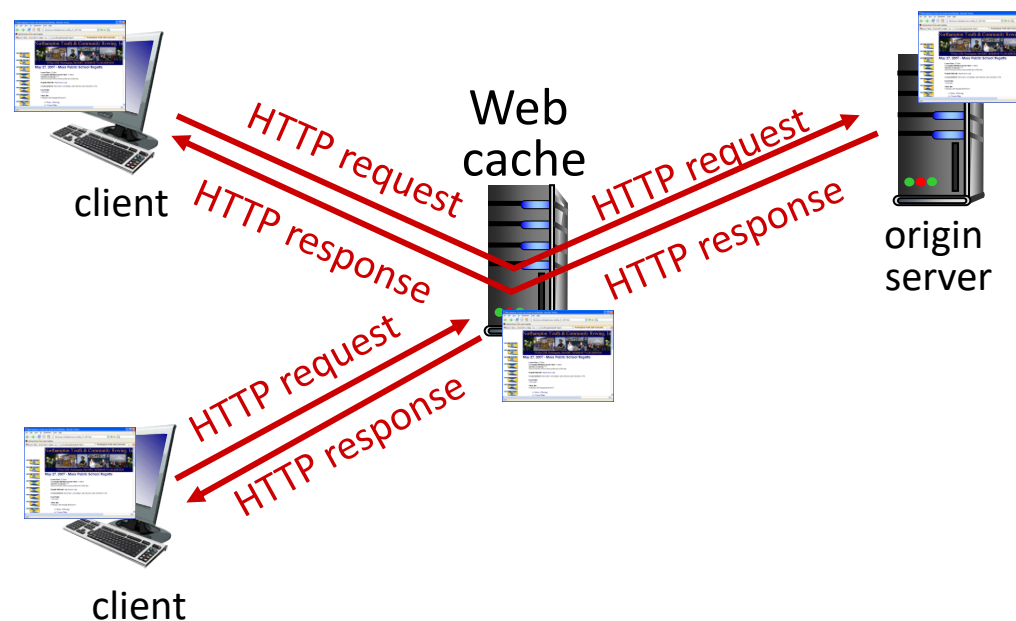
- Cookies允许站点知道太多关于用户的信息.
- 第三方持久cookie (跟踪 cookie) 允许跨多个网站跟踪公共标识 (cookie值)

Web Caches 缓存



Goal: 不访问原始服务器，就满足客户的请求

- 用户设置浏览器：通过**缓存**访问Web
- 浏览器将所有HTTP请求发送到缓存
 - 在缓存中的对象：缓存直接返回对象
 - 如对象不在缓存，缓存请求原始服务器，然后再将对象返回给客户端



Web Caches 缓存 (aka proxy servers)



- 缓存既是客户端又是服务器

- 原始请求客户端的服务器

 - 客户端到源服务器

- 服务器在响应头中告诉缓存对象允许的缓存:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

为什么要使用Web缓存?

- 降低客户端的请求响应时间

 - cache is closer to client

- 可以大大减少一个机构内部网络与Internet接入链路上的流量

- 互联网大量采用了缓存caches

 - 使“较弱的”内容提供者能够更有效地交付内容

缓存示例

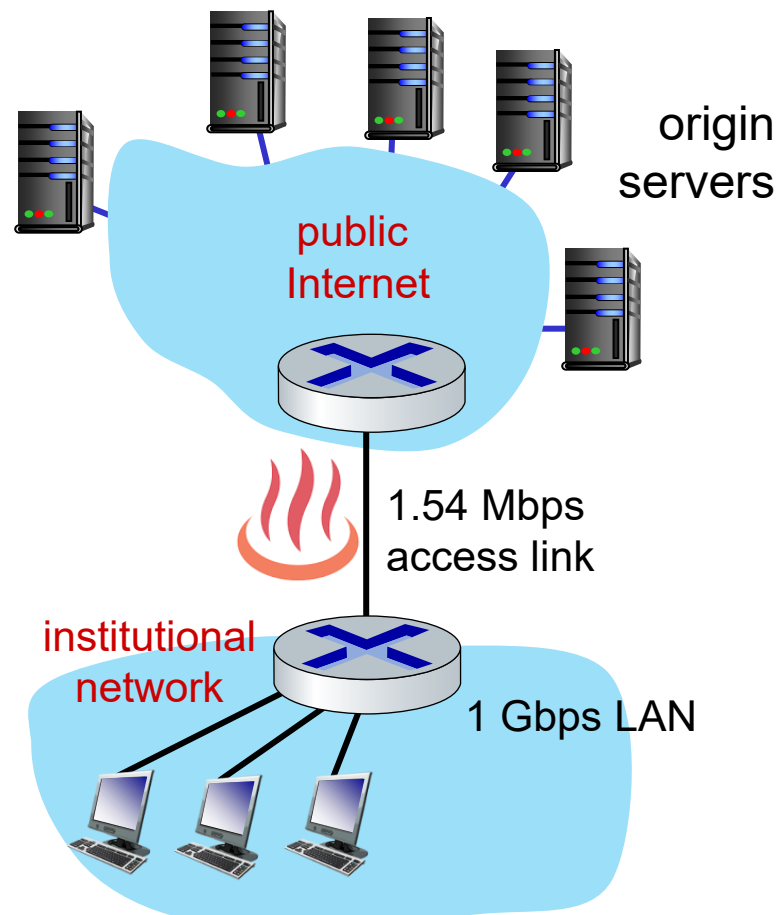
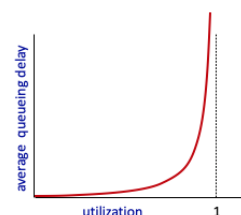


场景假设 Scenario:

- 接入链路带宽: 1.54 Mbps
- 从机构路由器到服务器的RTT: 2 sec
- 平均对象大小: 100K bits
- 机构内浏览器对原始服务器的平均请求率为: 15/sec
 - 平均到浏览器的速率: 1.50 Mbps

效果 Performance:

- 接入链路上的流量强度 = .97 *problem: large queueing delays at high utilization!*
- LAN的流量强度: .0015
- 总延时 = LAN延时 + 接入延时 + Internet 延时 = ms + 分 + 2s



缓存示例1：更快的接入链路



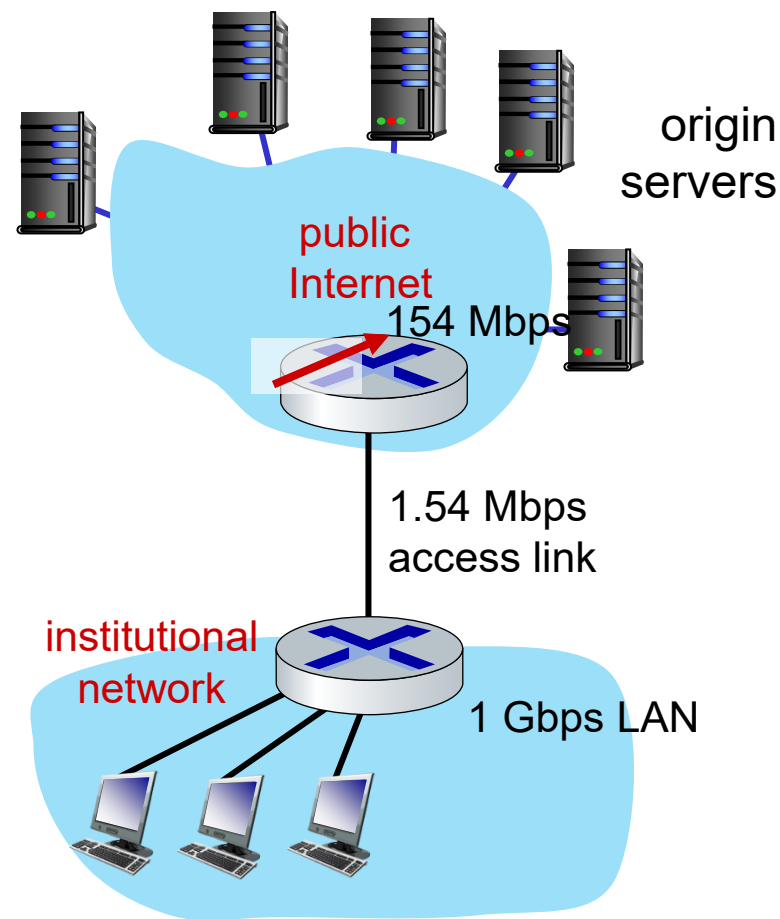
假设Scenario:

- 接入链路带宽: 1.54 Mbps
- 从机构路由器到服务器的RTT: 2 sec
- 平均对象大小: 100K bits
- 机构内浏览器对原始服务器的平均请求率为: 15/sec
 - 平均到浏览器的速率: 1.50 Mbps

效果 Performance:

- 接入链路上的流量强度 = .97 $\longrightarrow$.0097
- LAN的流量强度: .0015
- 总延时 = LAN延时 + 接入延时 + Internet
延时 = ms + 分 + 2s

Cost: faster access link (expensive!)



缓存示例2：安装本地缓存



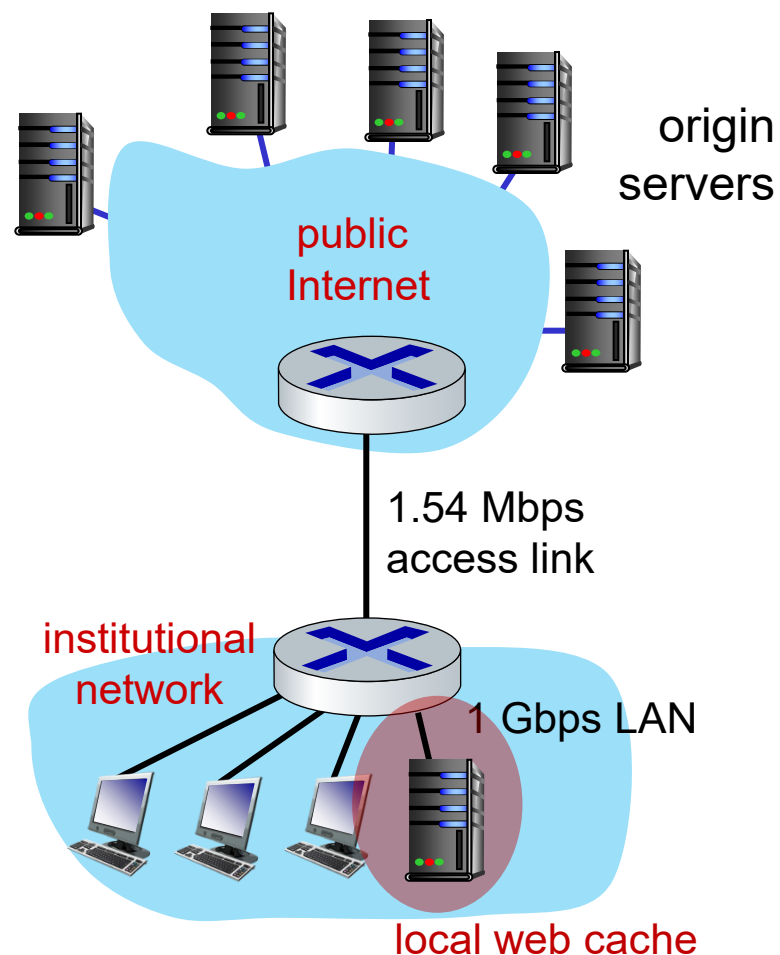
假设Scenario:

- 接入链路带宽: 1.54 Mbps
- 从机构路由器到服务器的RTT: 2 sec
- 平均对象大小: 100K bits
- 机构内浏览器对原始服务器的 平均请求率为: 15/sec
 - 平均到浏览器的速率: 1.50 Mbps

Cost: web cache (cheap!)

Performance: 如何计算链路利用率、延时?

- LAN 利用率: .?
- 接入网络利用率 = ?
- 总体延时 = ?



计算访问链路利用率，带缓存的端到端延迟：



假设缓存命中率：

- 40%请求在缓存中被满足, with low delay (msec)

- 其他60%的请求需要被原始服务器满足

- 经过接入链路到达浏览器的数据速率

$$= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$$

- 接入链路利用率 = $0.9 / 1.54 = .58$

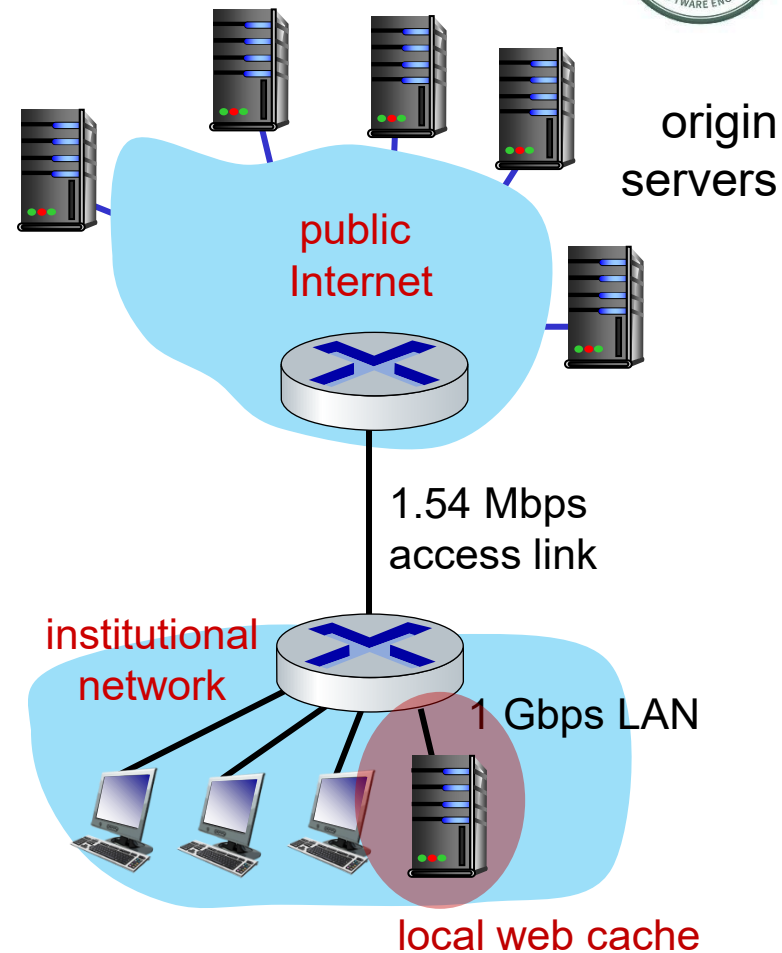
- 表示访问链路上的低排队延迟(msec)

- 总体延迟：

- = $0.6 * (\text{从原始服务器获取对象的延迟})$

- + $0.4 * (\text{从缓存获取对象的延迟})$

- = $0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$



比安装154Mbps链路还来得小 (而且比较便宜!)

条件GET方法



- **Goal:**如果缓存器中的对象备份是最新的，就不要发送对象

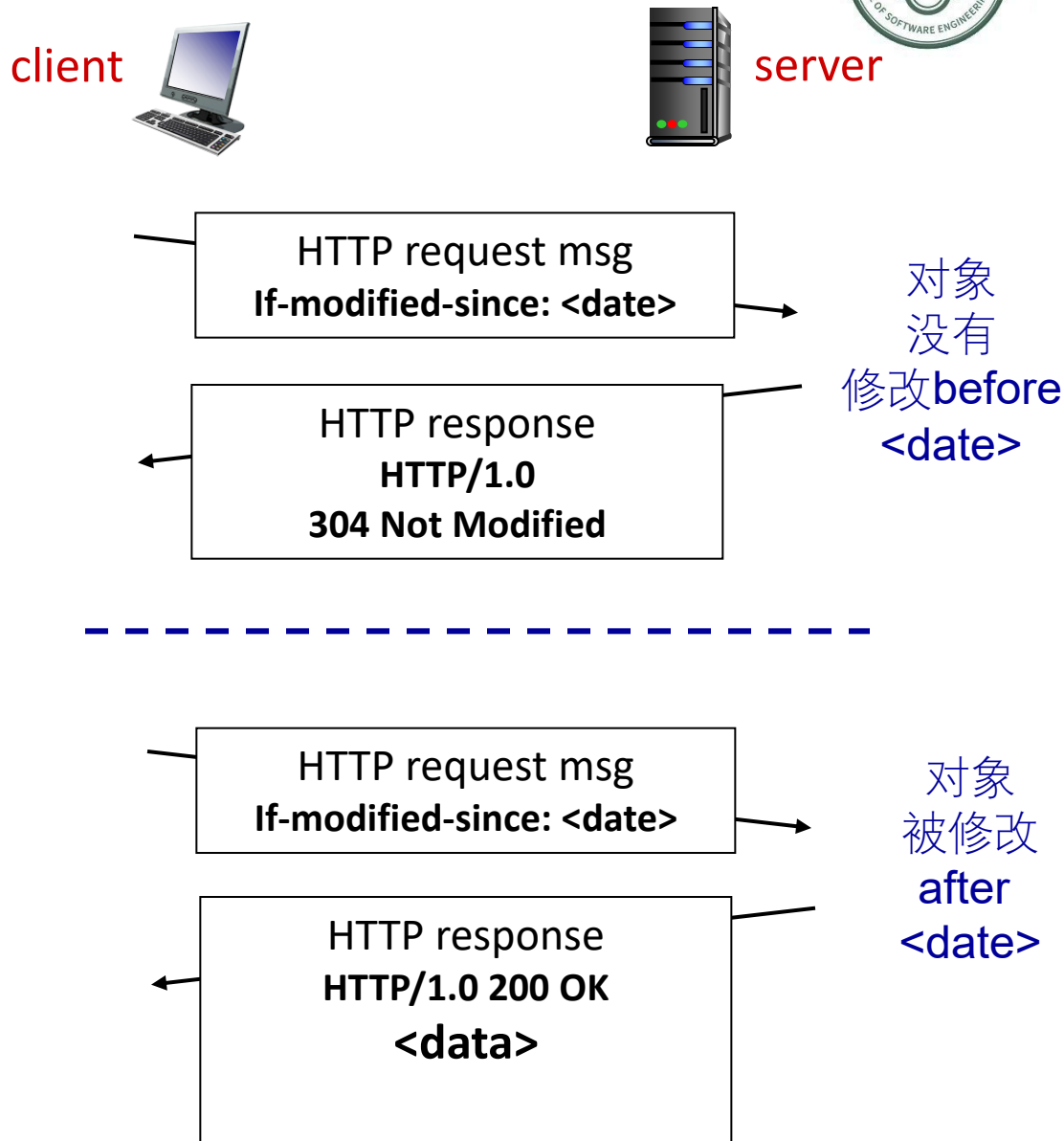
- 无对象传输延迟 (或使用网络资源)

- **缓存器:** 在HTTP请求中指定缓存备份的日期

- If-modified-since: <date>

- **服务器server:** 如果缓存备份未更新，则响应报文不包含对象:

- HTTP/1.0 304 Not Modified



Key goal: 减少多对象HTTP请求的延迟

HTTP1.1: 在单个TCP连接上引入了多个流水线 get (pipelined GETs)

- 服务器按顺序(FCFS:先到先服务调度)响应GET请求: 服务器按照GET请求顺序返回
- with FCFS, small object may have to wait for transmission (head-of-line (HOL) blocking) behind large object(s)使用FCFS, 小对象可能不得不等待大对象后面的传输 (头部阻塞)。
- 丢失恢复(重传丢失的TCP段), 暂停对象传输

Key goal: 减少多对象HTTP请求的延迟

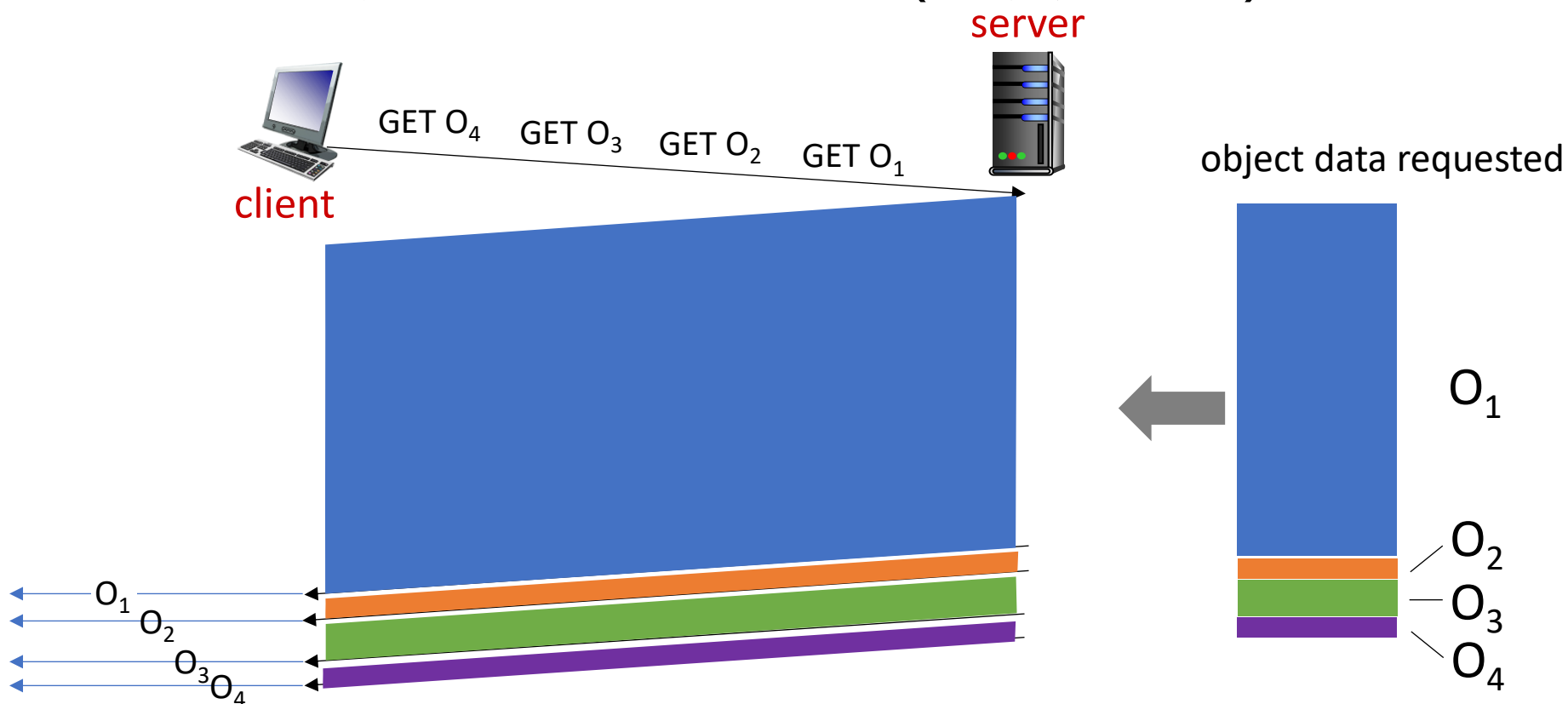
HTTP/2: [RFC 7540, 2015]增加了服务器向客户端发送对象的灵活性:

- 方法、状态码, 大多数头部字段与HTTP 1.1相同
- 基于客户端指定的对象优先级的请求对象的传输顺序(不一定是FCFS)
- 向客户端推送未请求的对象
- 将对象划分为帧, 调度帧以缓解HOL阻塞

HTTP/2:缓解HOL阻塞



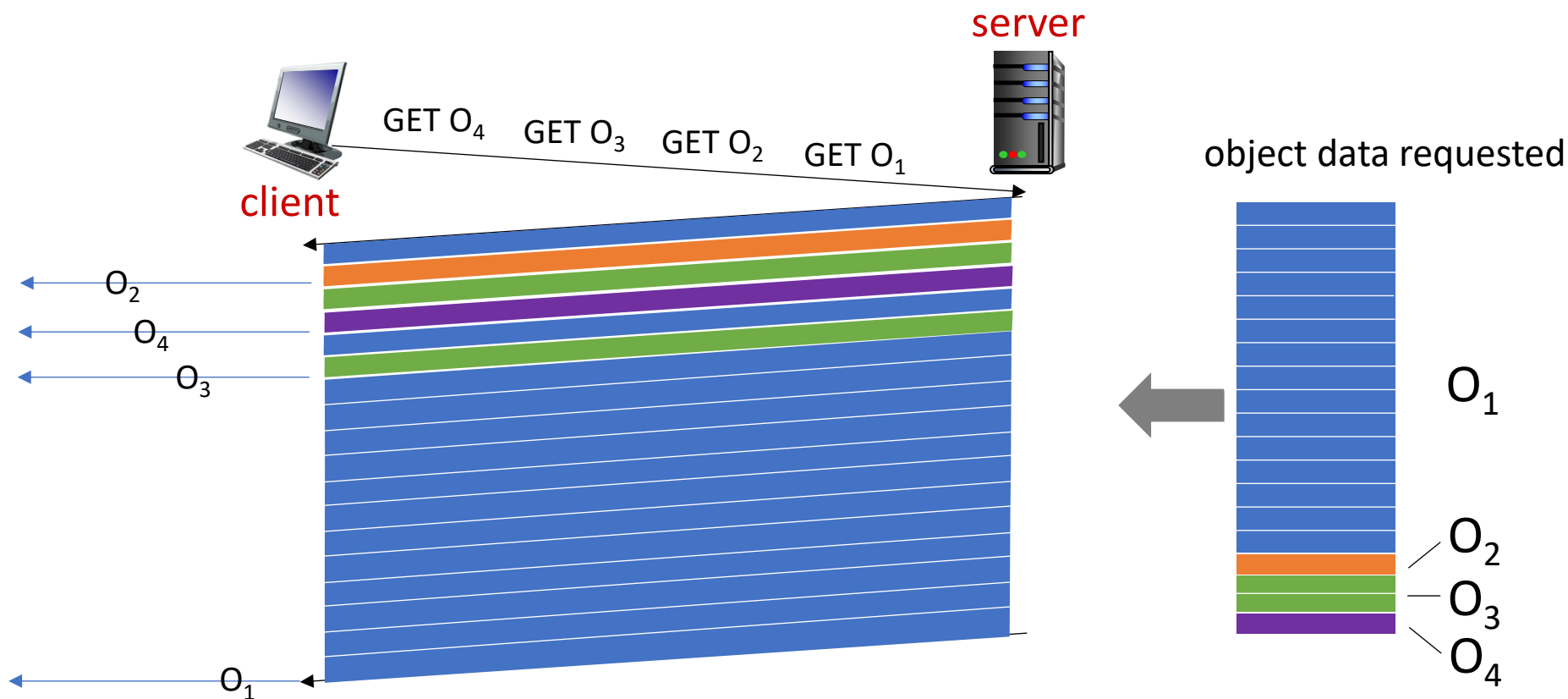
- HTTP 1.1:客户端请求1个大对象(如视频文件)和3个小对象



HTTP/2:缓解HOL阻塞



■ HTTP/2:对象分帧, 帧间穿插传送



O₂, O₃, O₄ delivered quickly, O₁ slightly delayed

HTTP/2 to HTTP/3

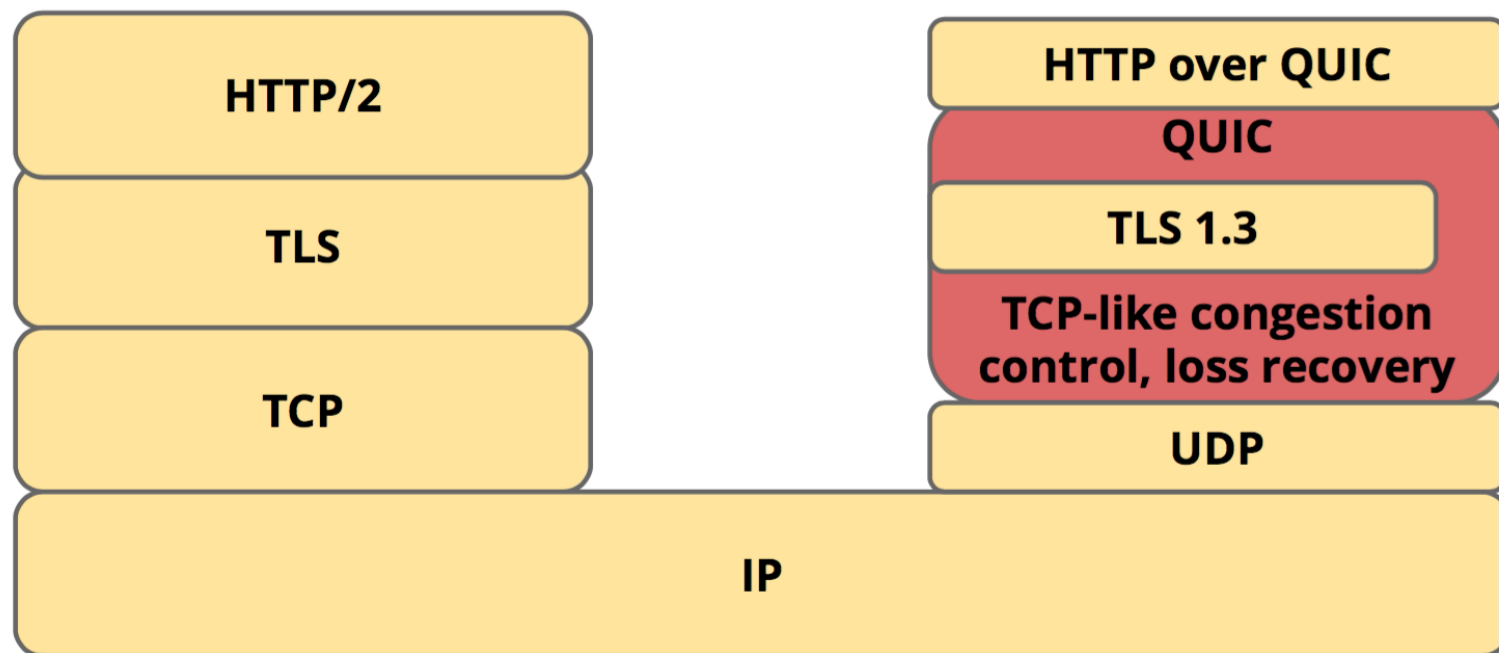


HTTP2解决了HTTP的队头阻塞问题，但是并没有解决TCP队头阻塞问题

HTTP/2通过单TCP连接的方式:

- 从数据包丢失恢复仍然使所有对象传输暂停
 - 同一个域名只是用一个TCP连接，TCP头部阻塞问题会更大
- **HTTP/3**: 在UDP上增加了安全性，每个对象的错误和拥塞控制（更多的管道）
 - 更多关于HTTP/3传输层的信息

HTTP3



- 基于Google的QUIC 协议
 - Quick UDP Internet Connection
- 基于 UDP 实现

- Check more info on:
<https://www.upyun.com/tech/tags/HTTP3>



中山大學

SUN YAT-SEN UNIVERSITY

软件工程学院

SCHOOL OF SOFTWARE ENGINEERING



谢谢

Yuhong Nan

nanyh@mail.sysu.edu.cn